Ollscoil Teicneolaíochta an Oirdheiscirt
South East Technological University

AI PRINCIPLES AND PRACTICE · WEEK 8

The Mathematics That Matters

Core booklet: shape, surprise and space

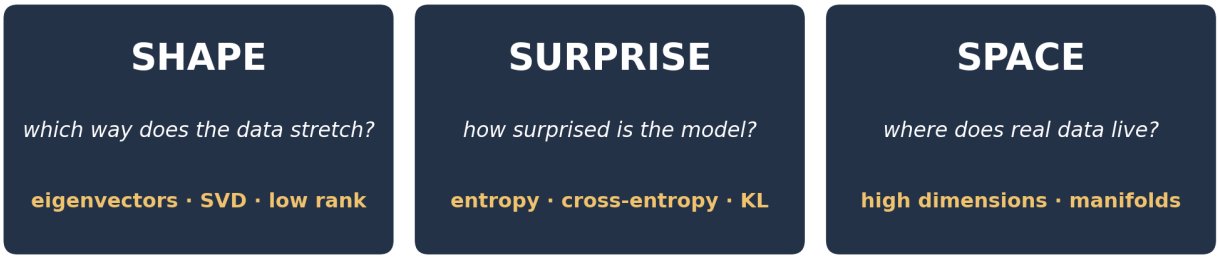
Plain English first. Every formula written three ways: in symbols, in words, and with numbers you can check on your phone.

Lecturer: Indrakshi Dey

How to use this booklet

This booklet goes with the Week 8 lecture and lab. It follows the lecture's three parts in the same order, with the same examples, and slows each one down: every picture is explained, every formula is written three ways, and every number is worked through so you can check it yourself.

You have already used all three ideas. In Week 5, PCA found the way your data stretches most. In Week 7, the loss measured how surprised the network was by the right answer. In Week 9, LoRA will change a huge model using two thin strips of numbers. This week you see what is underneath each one.

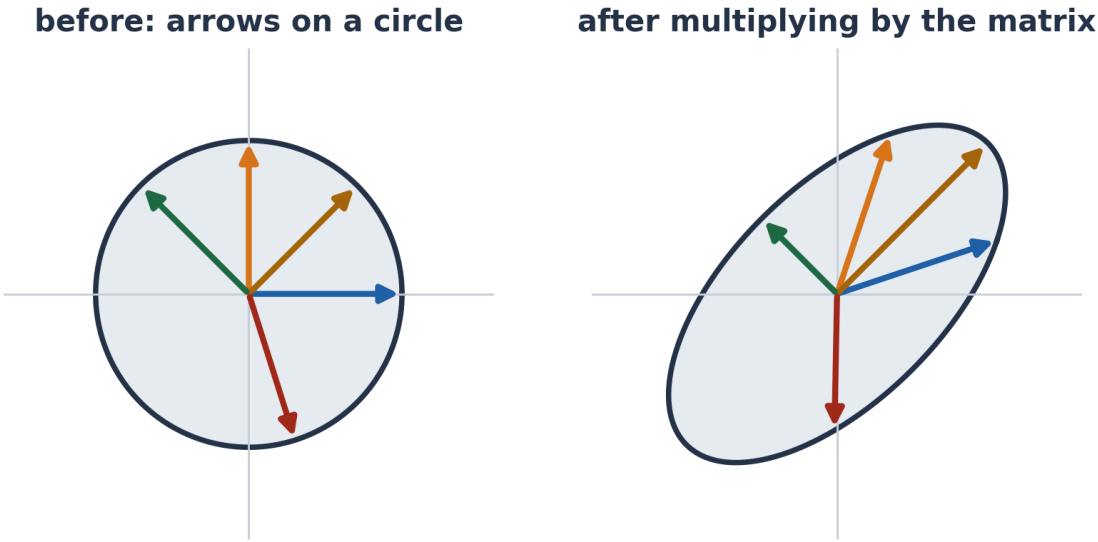


> **KEY IDEA**
> Three questions explain why the methods you have used work. **Shape:** which way does the data stretch? **Surprise:** how surprised is the model? **Space:** where does real data live?

Each section has the same boxes. **In plain English** says the idea with no symbols. **Everyday analogy** ties it to something familiar. The formula box writes it three ways. **Worked example** does the arithmetic. **Keep this** marks what to remember. Grey **Going further** boxes are optional.

Part 1 Shape: which way does the data stretch?

1.1 A table of numbers can move points



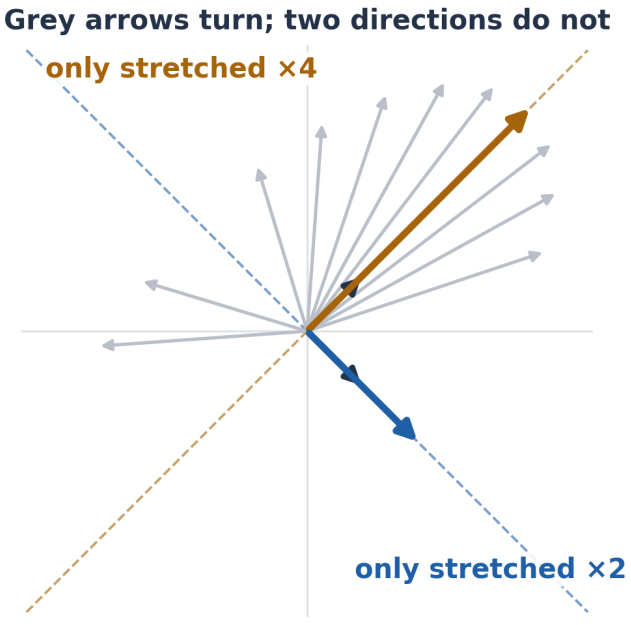
IN PLAIN ENGLISH

A matrix is a table of numbers. Multiplying an arrow by it gives a new arrow: usually longer or shorter, and pointing somewhere else. Every arrow on the circle on the left moves to the oval on the right.

EVERYDAY ANALOGY

A fairground mirror. It stretches your reflection more in some directions than others, and the picture comes out bent.

1.2 Some directions only get longer: eigenvectors



Most arrows are turned. But a few special directions come out pointing exactly the same way: they are only stretched. Those directions are the matrix's **eigenvectors**, and how much each one is stretched is its **eigenvalue**.

THE FORMULA $$Av = \lambda v$$

IN WORDS **matrix × special direction = stretch factor × the same direction**

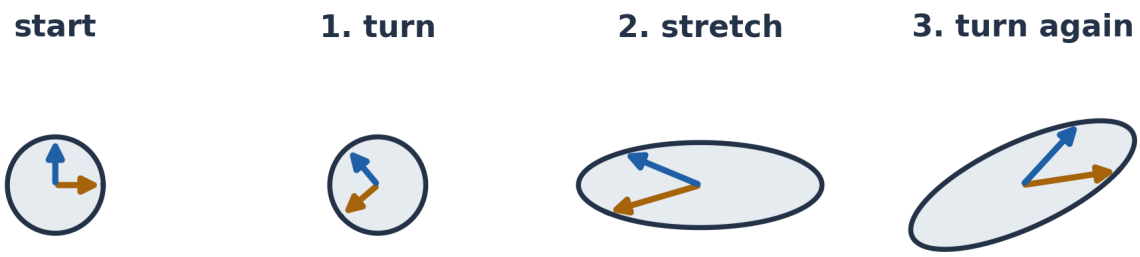
WITH NUMBERS **[[3, 1], [1, 3]] · (1, 1) = (4, 4) = 4 · (1, 1)**

Worked example: which arrows keep their direction?
A = [[3, 1], [1, 3]]. To multiply, each new number is a row of A times the arrow.
- (1, 0) → (3 × 1 + 1 × 0, 1 × 1 + 3 × 0) = (3, 1). Turned.
- (1, 1) → (3 + 1, 1 + 3) = (4, 4) = 4 × (1, 1). Same direction, stretched ×4.
- (1, −1) → (3 − 1, 1 − 3) = (2, −2) = 2 × (1, −1). Same direction, stretched ×2.

A	the matrix
v	an eigenvector: a special direction
λ ("lambda")	its eigenvalue: the stretch factor

EVERYDAY ANALOGY
Stretching a rubber sheet by pulling two corners. Arrows drawn along the pull only get longer; every other arrow tilts.

1.3 Every matrix does three simple things: the SVD



Eigenvectors only exist for square matrices, and even then they can be awkward. The **singular value decomposition** (SVD) works for any table of numbers, of any shape. It says every matrix does just three things: turn, stretch, turn.

THE FORMULA $$A = U\Sigma V^T$$

IN WORDS **any matrix = turn × stretch × turn**

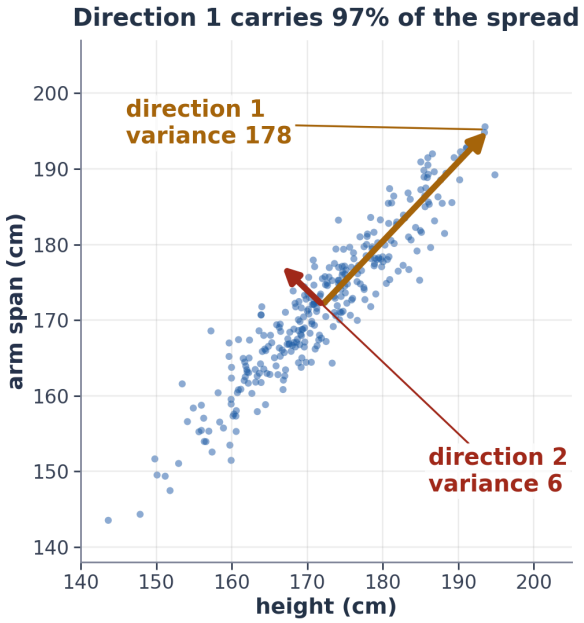
WITH NUMBERS **the matrix in the picture stretches by 2.54 one way and 0.88 the other**

V^T	the first turn
Σ ("sigma")	the stretches $\sigma_1, \sigma_2, \ldots$, biggest first: the **singular values**
U	the second turn

★ KEEP THIS

The SVD lists the directions a table of numbers stretches most, biggest first. Everything in Part 1 uses that list.

1.4 PCA is the SVD of your data



Put your data in a table, one row per example and one column per feature. Subtract each column's average (**centring**), and take the SVD. The first row of V^T is the direction the data spreads most: exactly what PCA found in Week 5.

THE FORMULA $$X_c = U\Sigma V^T, \quad \text{variance}_k = \sigma_k^2/(n-1)$$

IN WORDS **centred data = turn × stretch × turn; the biggest stretch is the main direction**

WITH NUMBERS **height and arm span of 300 people: variances 177.6 and 5.7, so 97% of the spread lies along one direction**

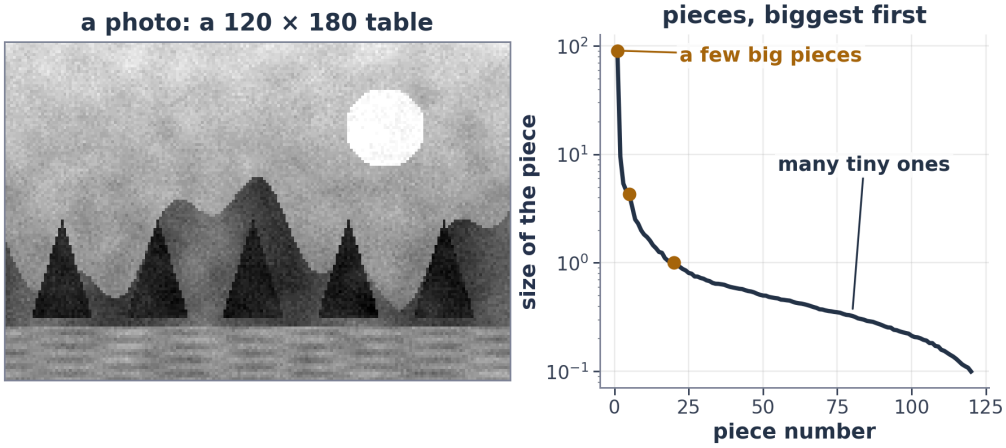
Worked example: PCA via SVD in five steps

These are the five lines you write in the lab, and reuse in your capstone.

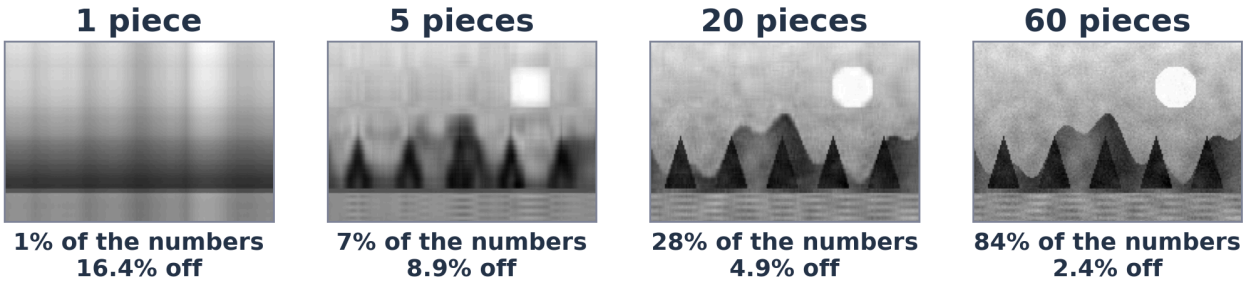
1. Centre: `Xc = X - X.mean(0)`.
2. Take the SVD: `U, S, Vt = np.linalg.svd(Xc, full_matrices=False)`.
3. The top k directions are the first k rows of `Vt`.
4. The spread along each is `S[:k] ** 2 / (n - 1)`.
5. The data in the new directions is `Xc @ components.T`.

For the height and arm span cloud: direction 1 is about (0.69, 0.72), "height and arm span in about equal amounts", and carries 177.6 ÷ (177.6 + 5.7) = 97% of the spread.

1.5 A photo is a table too: keep the big pieces



A black-and-white photo is a table of brightness values. Its SVD splits it into pieces, each one direction times a stretch, biggest first. A few pieces are big; most are tiny.

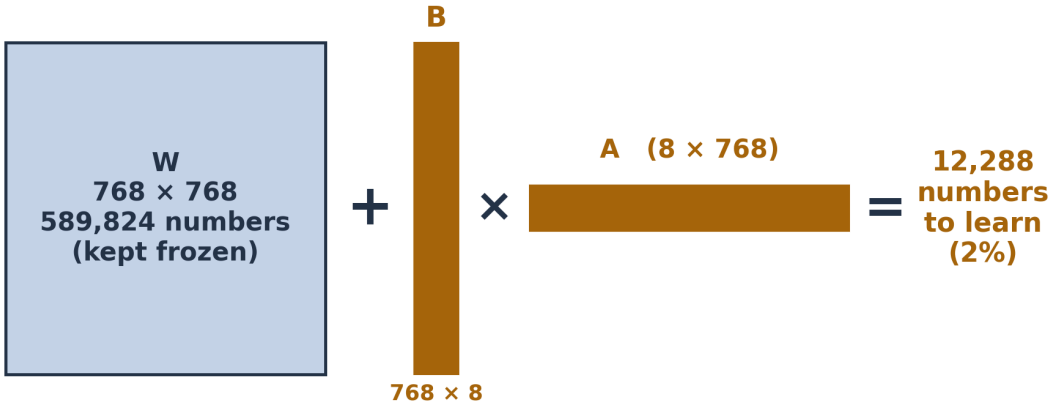


THE FORMULA	$A \approx \sigma_1 \boldsymbol{u}_1 \boldsymbol{v}_1^T + \ldots + \sigma_k \boldsymbol{u}_k \boldsymbol{v}_k^T$
IN WORDS	**picture ≈ biggest piece + next biggest + … (k pieces)**
WITH NUMBERS	**k = 20: 28% of the numbers, only 4.9% off**

Worked example: how many numbers does k = 20 store?
Each piece needs one column of U (120 numbers), one row of V^T (180 numbers) and one stretch: 301 numbers. Twenty pieces: 20 × 301 = 6,020 numbers, against 120 × 180 = 21,600 for the whole photo. That is 28%.

IN PLAIN ENGLISH
Keeping only the biggest pieces is a **low-rank approximation**. No other approximation with k pieces gets closer (this is the Eckart–Young theorem).

1.6 LoRA: change a huge table with two thin strips



Next week you fine-tune a large pretrained model. Changing all its weights is expensive. LoRA bets that the **change** is low-rank: it can be written as a tall thin strip times a short wide strip.

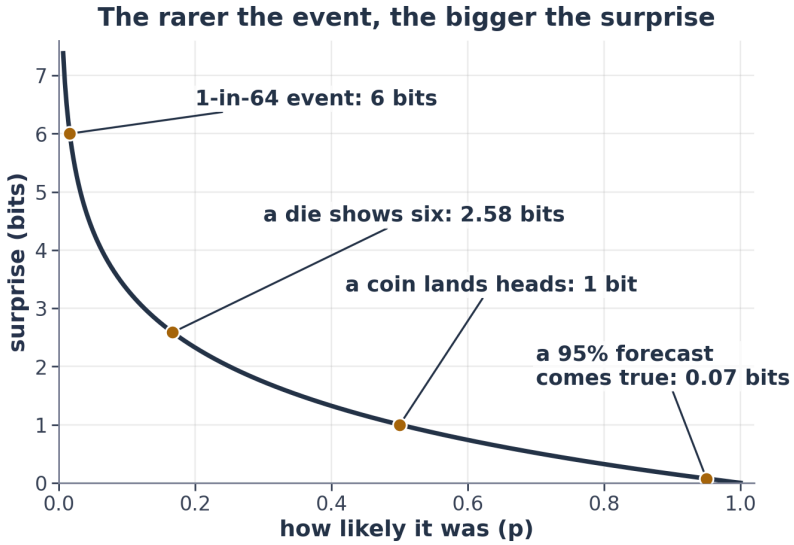
THE FORMULA	$W' = W + BA$
IN WORDS	**new weights = old weights (kept frozen) + thin strip × thin strip**
WITH NUMBERS	**a 768 × 768 layer, rank 8: 12,288 numbers to learn instead of 589,824 (about 2%)**

⋆ KEEP THIS

Shape in one line: the SVD finds the directions that matter, biggest first; keeping a few is a low-rank approximation, and LoRA uses the same idea to fine-tune big models cheaply.

Part 2 Surprise: how surprised is the model?

2.1 Rare things are surprising



IN PLAIN ENGLISH

If something likely happens, you learn little. If something rare happens, you learn a lot. Surprise measures that: the rarer the event, the bigger the surprise.

THE FORMULA $$\text{surprise} = -\log_2 p$$

IN WORDS **surprise = minus the log of how likely it was**

WITH NUMBERS **a coin: $-\log_2(1/2) = 1$ bit · a 1-in-64 event: 6 bits**

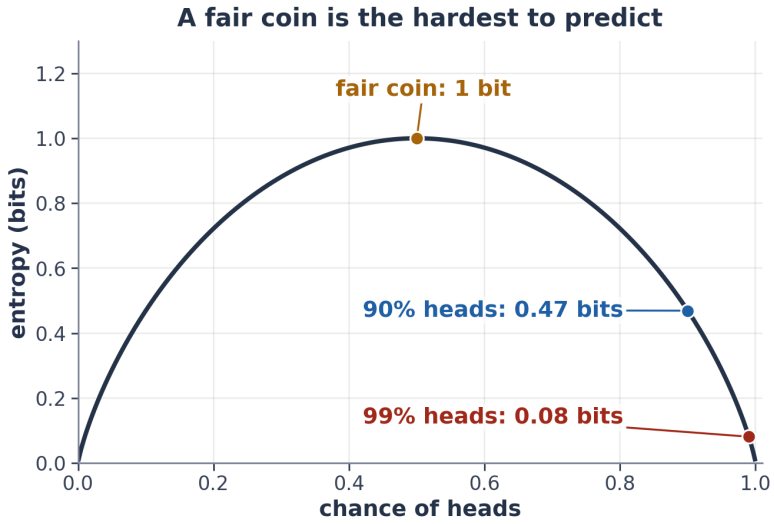
Worked example: three surprises

- A coin lands heads: p = 1/2, surprise = $\log_2 2 = 1$ bit.
- A die shows six: p = 1/6, surprise = $\log_2 6 \approx 2.58$ bits.
- A 1-in-64 event: surprise = $\log_2 64 = 6$ bits, because $2 \times 2 \times 2 \times 2 \times 2 \times 2 = 64$.

GOING FURTHER · OPTIONAL · BITS AND NATS

With log base 2 the unit is the **bit**. PyTorch and scikit-learn use the natural log (ln), so their losses are in **nats**. To convert, divide nats by ln 2 = 0.693: 1 nat ≈ 1.44 bits.

2.2 Entropy: the average surprise



Entropy is how surprised you are **on average**: each outcome's surprise, weighted by how often it happens. A fair coin is the hardest coin to predict, so it has the most entropy.

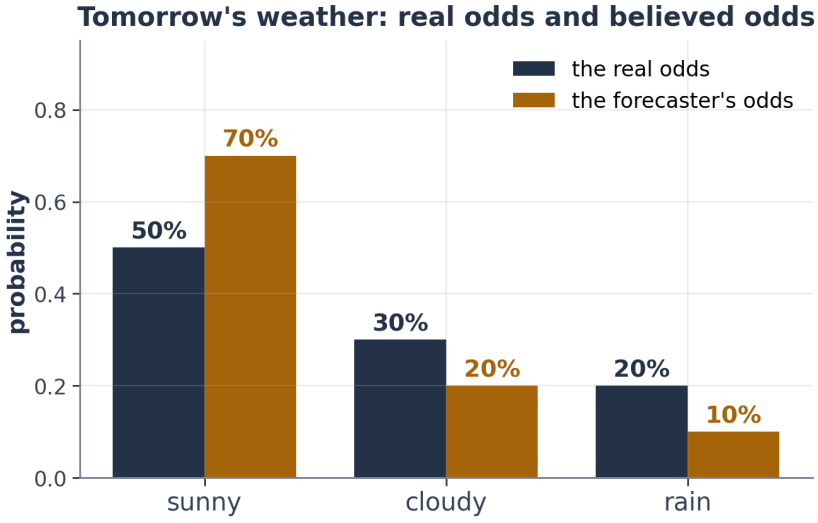
THE FORMULA $$H = -\sum p \log_2 p$$

IN WORDS **entropy = the surprise of each outcome, averaged by how often it happens**

WITH NUMBERS **fair coin: 1 bit · a 90/10 coin: 0.47 bits**

> **Worked example: a coin that lands heads 90% of the time**
> Heads: 0.9 × (−log₂ 0.9) = 0.9 × 0.152 = 0.137. Tails: 0.1 × (−log₂ 0.1) = 0.1 × 3.32 = 0.332. Entropy = 0.137 + 0.332 = **0.47 bits**: much easier to predict than a fair coin.

2.3 Cross-entropy: believing the wrong odds



EVERYDAY ANALOGY

A weather forecaster. The real odds for tomorrow are 50% sun, 30% cloud, 20% rain. A forecaster who believes 70/20/10 will be more surprised by the real weather, on average, than one who believes the real odds.

THE FORMULA $$H(p, q) = -\sum p \log_2 q$$

IN WORDS **cross-entropy = average surprise when the real odds are p but you believe q**

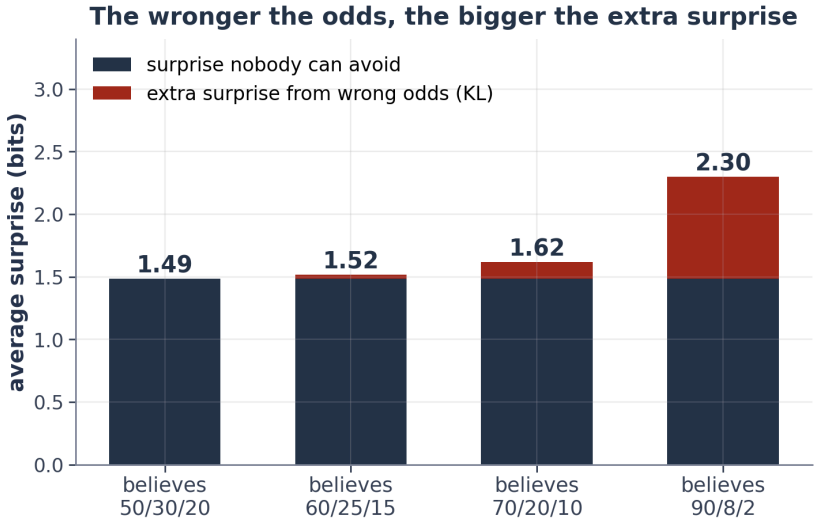
WITH NUMBERS **believing 70/20/10 when the truth is 50/30/20: 1.62 bits instead of 1.49**

Worked example: the forecaster's average surprise

The real odds decide how often each outcome happens; the forecaster's odds decide how surprising it feels.

- Entropy (believing the truth): 0.5 × 1 + 0.3 × 1.737 + 0.2 × 2.322 = 0.500 + 0.521 + 0.464 = **1.485 bits**.
- Cross-entropy: 0.5 × (−$\log_2$ 0.7) + 0.3 × (−$\log_2$ 0.2) + 0.2 × (−$\log_2$ 0.1) = 0.257 + 0.697 + 0.664 = **1.618 bits**.

2.4 KL divergence: the extra cost of wrong odds



THE FORMULA $$D_{\mathrm{KL}} = H(p, q) - H(p)$$

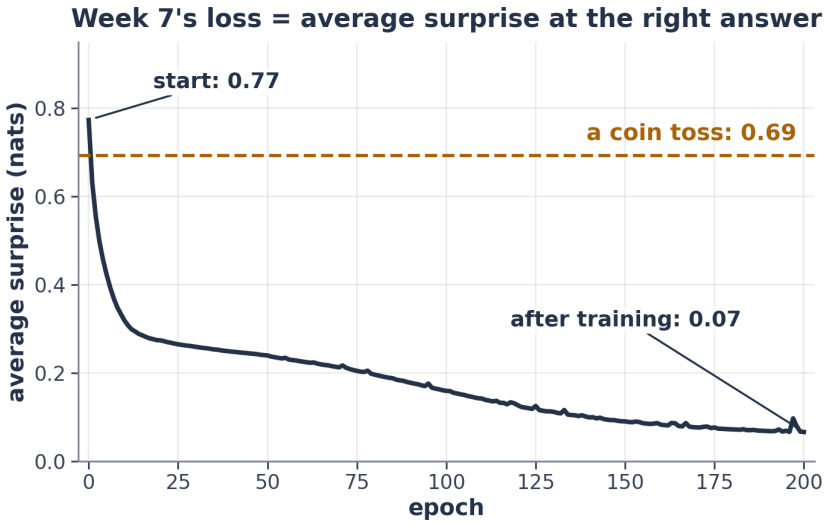
IN WORDS **KL = the extra surprise you pay for believing the wrong odds**

WITH NUMBERS **1.62 − 1.49 = 0.13 bits**

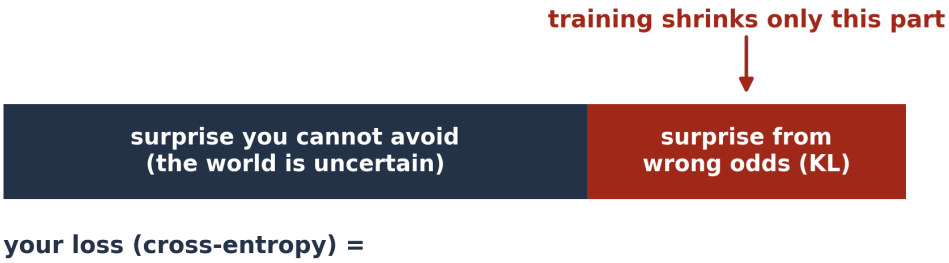
★ KEEP THIS

KL is never below zero, and is zero only when you believe the real odds. It is not a distance: the cost of believing q when the truth is p differs from the cost of believing p when the truth is q.

2.5 Your loss was the average surprise all along



In Week 7 the loss for the moons network fell from 0.77 to 0.07. That number is the network's average surprise (in nats) at the right answer. A network that guessed 50/50 every time would score ln 2 = 0.69: a coin toss.



IN PLAIN ENGLISH

Your loss (cross-entropy) is surprise you cannot avoid, because the world is uncertain (entropy), plus surprise from the model's wrong odds (KL). Training can only shrink the second part, so it pulls the model's odds towards the real ones. That is why cross-entropy is the right loss for probabilities.

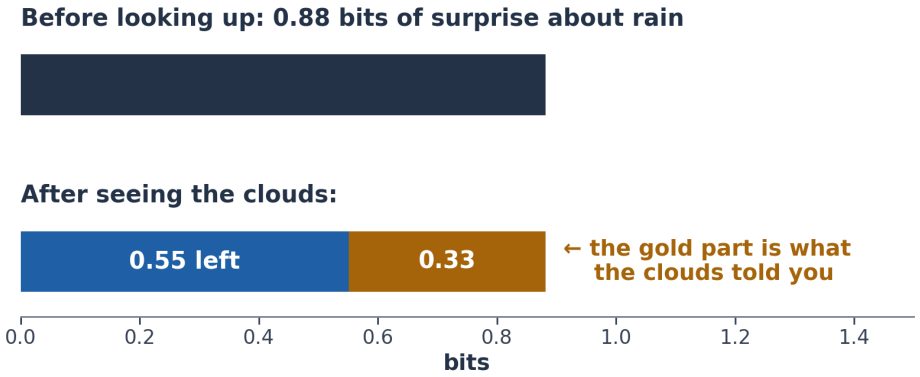
Worked example: a real model on the credit data

Logistic regression's average surprise at the right answer is 0.472 nats. A model that only knows "22% of customers default" pays 0.528 nats: the entropy of the labels. The 23 features save 0.056 nats, about 0.08 bits per customer. They help, but only a little, which matches Week 2: accuracy 0.81 against 0.78 for predicting "no default" for everyone.

GOING FURTHER · OPTIONAL · GOING FURTHER: MAXIMUM LIKELIHOOD

Adding up −log q(right answer) over the data is the same as −log of the product of those probabilities. So minimising cross-entropy is the same as choosing the dials that make the observed answers most likely: **maximum likelihood**.

2.6 What one thing tells you about another: mutual information

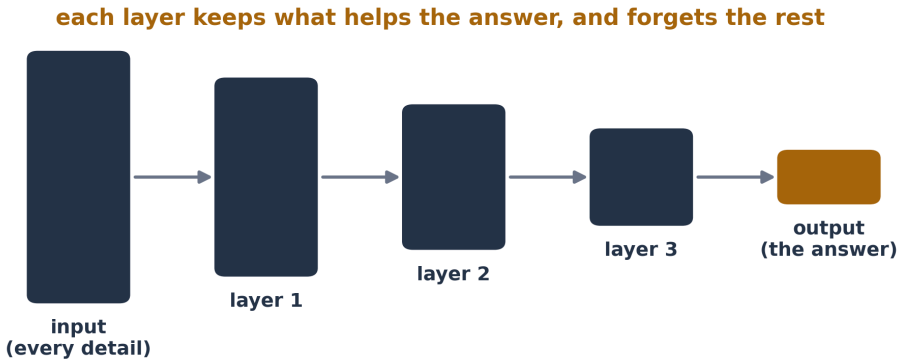


Rain falls on 30% of days. Dark clouds appear on 90% of rainy days and 20% of dry ones. Before you look up, you are 0.88 bits uncertain about rain; after seeing the clouds, only 0.55. The clouds told you 0.33 bits.

THE FORMULA $$I = H(\text{before}) - H(\text{after})$$

IN WORDS **what the clouds tell you = surprise about rain before looking − surprise after looking**

WITH NUMBERS **0.881 − 0.551 = 0.33 bits**



IN PLAIN ENGLISH

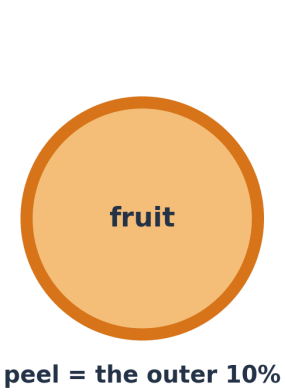
The **information bottleneck** is a useful way to picture hidden layers: each layer keeps what helps predict the answer and forgets the rest of the input's detail.

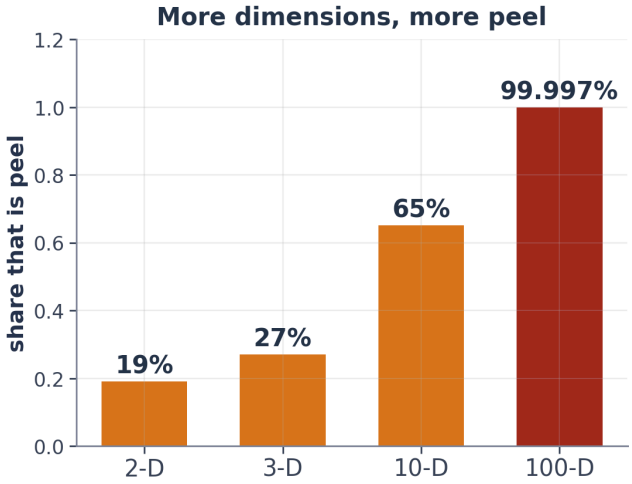
⋆ KEEP THIS

Surprise in one line: your loss is unavoidable surprise plus avoidable surprise (KL), and training shrinks the avoidable part.

Part 3 Space: where does real data live?

3.1 In high dimensions, an orange is almost all peel





THE FORMULA

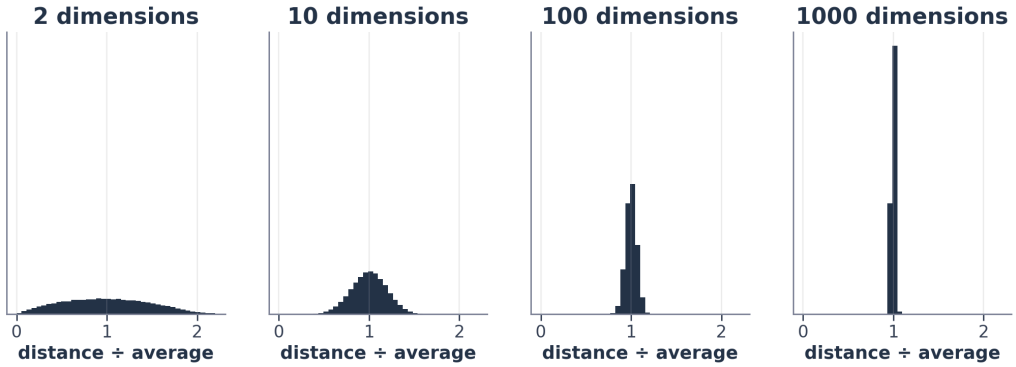
$$\text{peel} = 1 - 0.9^{d}$$

IN WORDS **peel = 1 − (0.9 × 0.9 × …, once for every dimension)**

WITH NUMBERS **2-D: 19% · 10-D: 65% · 100-D: 99.997%**

Worked example: 10 dimensions
In each dimension the fruit keeps 90% of the width. In 10 dimensions: 0.9 multiplied by itself 10 times is 0.349, so the fruit is 35% of the orange and the peel is 65%.

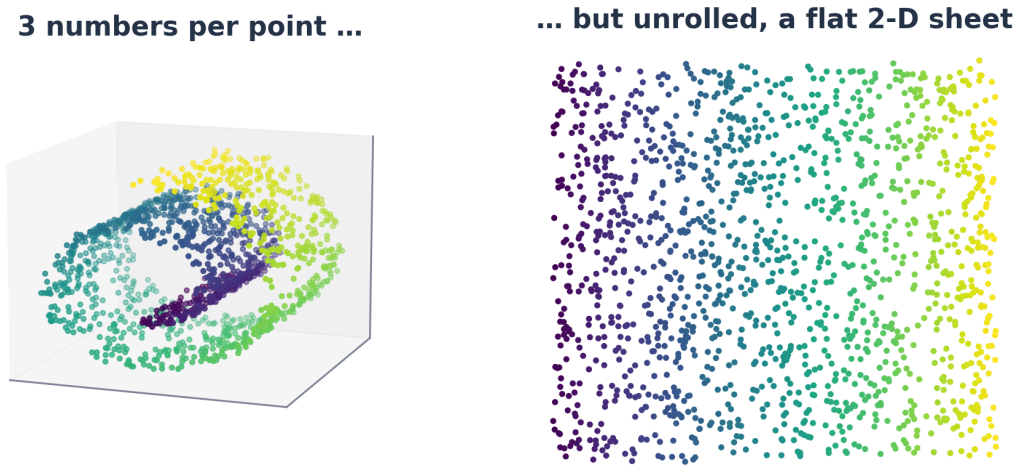
3.2 Everyone is about equally far away



IN PLAIN ENGLISH

With 400 random points, the farthest pair is 593 times farther apart than the nearest pair in 2 dimensions, but only 1.19 times in 1,000 dimensions. When everyone is about equally far away, “nearest neighbour” stops meaning much. This is the **curse of dimensionality**, and it is why distance-based methods struggle on raw high-dimensional data.

3.3 Real data is a rolled-up sheet: the manifold hypothesis

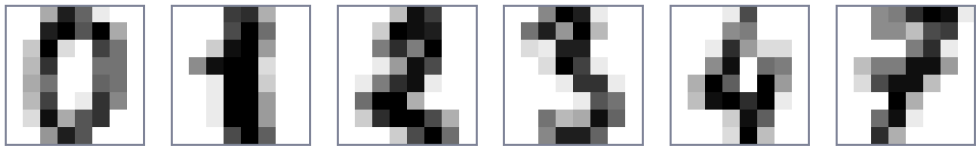


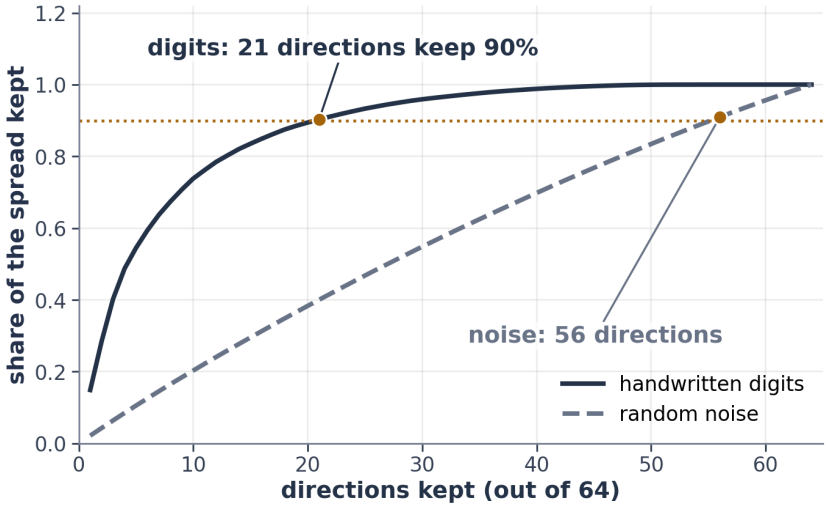
EVERYDAY ANALOGY

A sheet of paper rolled up in a room. Each point needs three numbers to say where it is in the room, but really it only needs two: where it is on the sheet.

The **manifold hypothesis** says real data is like this: it lives near a thin, low-dimensional surface inside the space it is written in. That is why the curse of dimensionality does not ruin machine learning: real data does not fill the whole space.

3.4 Counting the directions that matter





THE FORMULA

$$\text{smallest } k \text{ with } \frac{\sigma_1^2 + \ldots + \sigma_k^2}{\sigma_1^2 + \ldots + \sigma_n^2} \geq 0.9$$

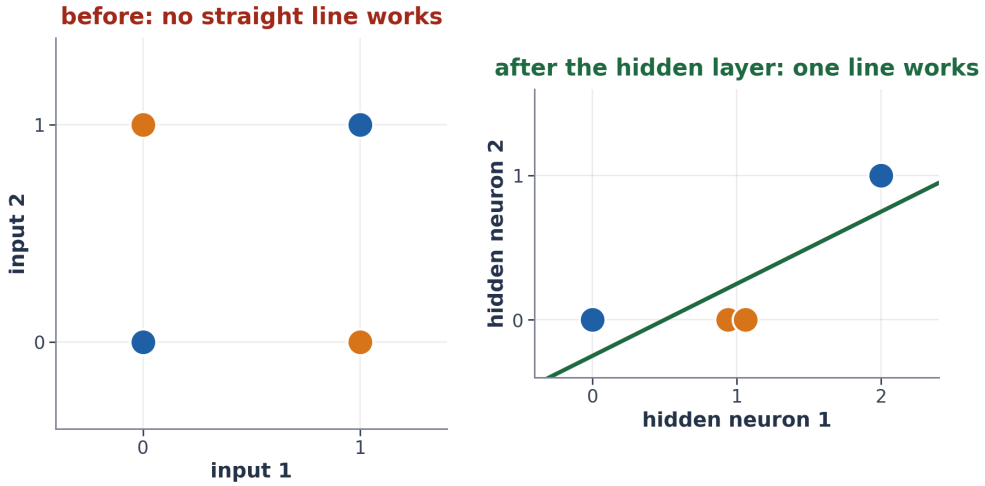
IN WORDS **how many directions you need to keep 90% of the spread**

WITH NUMBERS **handwritten digits: 21 of 64 · random noise: 56 of 64**

IN PLAIN ENGLISH

This rough count is the data's **intrinsic dimension**. Handwritten digits are written in 64 numbers but need about 21 directions; random noise needs nearly all of them, because noise has no sheet to live on. In the lab you count this for your own project data.

3.5 What a hidden layer does



In Week 7 the hidden layer moved the dots until a straight line could separate them. In the language of this week: it flattened the curved sheet the data lives on.

★ KEEP THIS

Space in one line: in many dimensions everything is near the edge and all distances look alike, but real data lives on a thin, low-dimensional sheet, and models work by finding it.

Part 4 Going further (optional)

GOING FURTHER · OPTIONAL · EIGENVECTORS AND THE SVD ARE RELATED

The SVD of a centred data table X and the eigenvectors of $X^T X$ (the covariance matrix, up to n − 1) give the same directions, and the eigenvalues are σ_k^2. That is why PCA can be done either way; the SVD is the more accurate on a computer.

GOING FURTHER · OPTIONAL · ANOTHER WAY TO COUNT DIMENSIONS: TWONN

The TwoNN estimator uses each point's distances to its nearest and second-nearest neighbours, r_1 and r_2: $\hat{d} = n / \sum \log(r_2 / r_1)$. For the handwritten digits it gives about 9, lower than the 21 directions from PCA, because the digits' sheet is curved and PCA can only count straight directions.

GOING FURTHER · OPTIONAL · A CAUTION ABOUT THE INFORMATION BOTTLENECK

It is a useful picture, not a law: researchers still debate how exactly it applies to real networks.

Practice questions

Try each question before reading the answer below it.

1. In one sentence, what is an eigenvector?

A direction a matrix only stretches, without turning it.

2. A = [[2, 0], [0, 5]]. What are its eigenvectors and eigenvalues?

(1, 0) stretched ×2, and (0, 1) stretched ×5.

3. What three things does every matrix do, according to the SVD?

Turn, stretch, turn: $A = U\Sigma V^T$.

4. Why must you centre the data before taking the SVD for PCA?

Otherwise the first direction points at the data's average, not along its spread.

5. Two directions have variances 90 and 10. What share does the first carry?

90 ÷ 100 = 90%.

6. A 100 × 200 picture, k = 10 pieces. How many numbers are stored, and what share?

10 × (100 + 200 + 1) = 3,010 of 20,000: about 15%.

7. A rank-4 LoRA change to a 1,000 × 1,000 layer: how many numbers are learned?

2 × 1,000 × 4 = 8,000, instead of 1,000,000.

8. What is the surprise, in bits, of an event with probability 1/8?

$\log_2 8 = 3$ bits.

9. Which has more entropy: a fair coin or a coin that lands heads 99% of the time?

The fair coin (1 bit against 0.08 bits).

10. Cross-entropy is 2.0 bits and entropy is 1.5 bits. What is the KL divergence?

0.5 bits.

11. Can KL be negative?

No. It is zero only when the believed odds equal the real ones.

12. A loss of 0.69 nats on a two-class problem means what?

The model is no better than a coin toss: ln 2 = 0.693.

13. Before seeing X you are 1.0 bit uncertain about Y; after, 0.7 bits. What is the mutual information?

0.3 bits.

14. In 20 dimensions, what share of an orange is peel (outer 10%)?

$1 - 0.9^{20} = 1 - 0.12 = 88\%$.

15. Why does nearest-neighbour search struggle in very high dimensions?

All distances become about equal, so “nearest” carries little information.

16. Digits need 21 of 64 directions for 90%; noise needs 56. What does that tell you?

The digits live on a low-dimensional sheet; noise fills the space.

Formula cheat sheet

The formula	In words
$A\boldsymbol{v} = \lambda\boldsymbol{v}$	matrix × special direction = stretch factor × the same direction
$A = U\Sigma V^T$	any matrix = turn × stretch × turn
$X_c = U\Sigma V^T$	PCA: the SVD of the centred data; the spread along direction k is $\sigma_k^2/(n-1)$
$A \approx \sum_{i=1}^{k} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T$	low rank: keep the k biggest pieces
$W' = W + BA$	LoRA: frozen weights + thin strip × thin strip
$-\log_2 p$	surprise, in bits
$H = -\sum p \log_2 p$	entropy: the average surprise
$H(p, q) = -\sum p \log_2 q$	cross-entropy: average surprise with the wrong odds
$D_{\mathrm{KL}} = H(p, q) - H(p)$	KL: the extra surprise from wrong odds
$I = H(\text{before}) - H(\text{after})$	mutual information: what one thing tells you about another
$1 - 0.9^d$	the share of a d-dimensional orange that is peel

Glossary

bit, nat	Units of surprise: log base 2 gives bits; the natural log gives nats (1 nat ≈ 1.44 bits).
centring	Subtracting each column's average, so the data sits around zero.
cross-entropy	Average surprise when the real odds are p but you believe q; the loss for classification.
curse of dimensionality	In many dimensions, space is mostly edge and all distances look alike.
eigenvalue, eigenvector	A direction a matrix only stretches, and by how much.
entropy	The average surprise of an outcome; how unpredictable it is.
information bottleneck	The idea that layers keep what predicts the answer and forget the rest.
intrinsic dimension	How many directions data really needs, however many numbers it is written in.
KL divergence	The extra surprise from believing the wrong odds; never negative.
LoRA	Low-rank adaptation: fine-tuning by learning a thin-strip × thin-strip change.
low-rank approximation	Keeping only the biggest pieces of an SVD.
manifold hypothesis	Real data lies near a thin, low-dimensional surface.
mutual information	How much knowing one thing reduces your surprise about another.
PCA	Principal component analysis: the SVD of the centred data.
singular value	One of the stretches in the SVD, $\sigma_1 \geq \sigma_2 \geq \ldots$
SVD	Singular value decomposition: any matrix as turn × stretch × turn.
surprise	−log p: big for rare events, small for likely ones.